JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface UserDefinedFileAttributeView

All Superinterfaces:

AttributeView, FileAttributeView

---

public interface **UserDefinedFileAttributeView**
extends FileAttributeView

A file attribute view that provides a view of a file's user-defined attributes, sometimes known as *extended attributes*. User-defined file attributes are used to store metadata with a file that is not meaningful to the file system. It is primarily intended for file system implementations that support such a capability directly but may be emulated. The details of such emulation are highly implementation specific and therefore not specified.

This FileAttributeView provides a view of a file's user-defined attributes as a set of name/value pairs, where the attribute name is represented by a String. An implementation may require to encode and decode from the platform or file system representation when accessing the attribute. The value has opaque content. This attribute view defines the read and write methods to read the value into or write from a ByteBuffer. This FileAttributeView is not intended for use where the size of an attribute value is larger than Integer.MAX_VALUE.

The supportsFileAttributeView method may be used to test if a specific FileStore supports the storage of user-defined attributes.

Where dynamic access to file attributes is required, the getAttribute method may be used to read the attribute value. The attribute value is returned as a byte array (byte[]). The setAttribute method may be used to write the value of a user-defined attribute from a buffer (as if by invoking the write method), or byte array (byte[]).

Since:

1.7

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
void	**delete**(String name)	Deletes a user-defined attribute.
List<String>	**list**()	Returns a list containing the names of the user-defined attributes.

String	**name**()	Returns the name of this attribute view.
int	**read**(String name, ByteBuffer dst)	Read the value of a user-defined attribute into a buffer.
int	**size**(String name)	Returns the size of the value of a user-defined attribute.
int	**write**(String name, ByteBuffer src)	Writes the value of a user-defined attribute from a buffer.

Method Details

name

```
String name()
```

Returns the name of this attribute view. Attribute views of this type have the name `"user"`.

Specified by:

`name` in interface `AttributeView`

Returns:

the name of the attribute view

list

```
List<String> list()
           throws IOException
```

Returns a list containing the names of the user-defined attributes.

Returns:

An unmodifiable list containing the names of the file's user-defined

Throws:

`IOException` - If an I/O error occurs

size

```
int size(String name)
  throws IOException
```

Returns the size of the value of a user-defined attribute.

Parameters:

`name` - The attribute name

Returns:

The size of the attribute value, in bytes.

Throws:

`ArithmeticException` - If the size of the attribute is larger than `Integer.MAX_VALUE`

`IOException` - If an I/O error occurs

read

```
int read(String name,
         ByteBuffer dst)
  throws IOException
```

Read the value of a user-defined attribute into a buffer.

This method reads the value of the attribute into the given buffer as a sequence of bytes, failing if the number of bytes remaining in the buffer is insufficient to read the complete attribute value. The number of bytes transferred into the buffer is `n`, where `n` is the size of the attribute value. The first byte in the sequence is at index `p` and the last byte is at index `p + n - 1`, where `p` is the buffer's position. Upon return the buffer's position will be equal to `p + n`; its limit will not have changed.

Usage Example: Suppose we want to read a file's MIME type that is stored as a user-defined attribute with the name "`user.mimetype`".

```
    UserDefinedFileAttributeView view =
        Files.getFileAttributeView(path, UserDefinedFileAttributeView.class);
    String name = "user.mimetype";
    ByteBuffer buf = ByteBuffer.allocate(view.size(name));
    view.read(name, buf);
    buf.flip();
    String value = Charset.defaultCharset().decode(buf).toString();
```

Parameters:

`name` - The attribute name

`dst` - The destination buffer

Returns:

The number of bytes read, possibly zero

Throws:

`IllegalArgumentException` - If the destination buffer is read-only

`IOException` - If an I/O error occurs or there is insufficient space in the destination buffer for the attribute value

See Also:

`size(String)`

write

```
int write(String name,
          ByteBuffer src)
   throws IOException
```

Writes the value of a user-defined attribute from a buffer.

This method writes the value of the attribute from a given buffer as a sequence of bytes. The size of the value to transfer is `r`, where `r` is the number of bytes remaining in the buffer, that is `src.remaining()`. The sequence of bytes is transferred from the buffer starting at index `p`, where `p` is the buffer's position. Upon return, the buffer's position will be equal to `p + n`, where `n` is the number of bytes transferred; its limit will not have changed.

If an attribute of the given name already exists then its value is replaced. If the attribute does not exist then it is created. It is implementation specific if a test to check for the existence of the attribute and the creation of attribute are atomic with respect to other file system activities.

Where there is insufficient space to store the attribute, or the attribute name or value exceed an implementation specific maximum size then an `IOException` is thrown.

Usage Example: Suppose we want to write a file's MIME type as a user-defined attribute:

```
    UserDefinedFileAttributeView view =
        Files.getFileAttributeView(path, UserDefinedFileAttributeView.class);
    view.write("user.mimetype", Charset.defaultCharset().encode("text/html"));
```

Parameters:

`name` - The attribute name

`src` - The buffer containing the attribute value

Returns:

The number of bytes written, possibly zero

Throws:

IOException - If an I/O error occurs

delete

```
void delete(String name)
     throws IOException
```

Deletes a user-defined attribute.

Parameters:

name - The attribute name

Throws:

IOException - If an I/O error occurs or the attribute does not exist

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.